

An Efficient Implementation of KYBER

Wenbo Guo^{ID}, Shuguo Li^{ID}, *Member, IEEE*, and Liang Kong^{ID}

Abstract—Quantum algorithms pose a huge threat to the current cryptosystems. In this article, we present a hardware implementation of CRYSTALS-KYBER which is one of the post-quantum cryptosystems based on the Module-LWE problem. Using the proposed modular reduction algorithm, modified modular adder and the reconfigurable data path, the design shares the computing resource for different polynomial related operations, and achieves higher degree of parallelism. Our design is implemented on a Xilinx Artix-7 FPGA. Compared with the leading hardware implementations, our design is more compact, the execution time is shorter, and it significantly consumes fewer registers.

Index Terms—Post-quantum cryptography, NTT, modular reduction, hardware implementation, CRYSTALS-KYBER.

I. INTRODUCTION

THE CLASSICAL public-key cryptography protocols are no longer secure under the attack of a quantum computer based on Shor's algorithm [1]. CRYSTALS-KYBER is one of the lattice-based cryptosystems which is based on Module Learning-with-Errors (Module-LWE) [6] problem, and this algorithm has entered the third round of Post-Quantum Cryptography (PQC) algorithm evaluation by the National Institute of Standards and Technology (NIST) [2].

Software implementation of KYBER on ARM Cortex-M4 is studied in [7] which proposed a memory-efficient architecture while optimizing the Number Theoretic Transform (NTT) by vector instructions to improve the performance. The advantage of software implementation is that it has a short design cycle and is very flexible, but the performance is relatively weak compared with hardware implementation. Software and hardware co-design is a compromise, and many brief had studied on this architecture, such as [8] and [9]. The pure hardware implementation proposed in [10] modified the algorithm to make KYBER suitable for FPGA. Reference [5] proposed a compact hardware implementation, and optimized NTT, point-wise multiplication (PWM) and sampling.

To the best of our knowledge, there are very few pure hardware implementations of KYBER. In this brief, we propose a fast and compact implementation of KYBER and evaluate our

design on FPGA platform. The contributions of this brief are listed below.

1. We propose a resource-efficient modular reduction algorithm for the modulus $q = 3329$ in KYBER.
2. We construct a reconfigurable data path which can perform NTT and inverse NTT (INTT), point-wise-multiplication (PWM) and other polynomial related operations. We also merge the " $\times \frac{1}{2}$ " operation in $\mathbb{Z}_p$ into the specially modified modular adder to reduce the depth of the pipeline.
3. We define a set of instructions to control the data path and the Keccak core in parallel, and the instructions compatible with our SRAM organization method to maximize the parallelism of KYBER.

The rest of this brief is organized as follows: In Section II, we briefly introduce the operations that related to KYBER. In Section III, our proposed architecture is described in detail. The experimental results are shown and discussed in Section IV. Finally, we conclude this brief in Section V.

II. BACKGROUND

A. Modular Reduction and NTT

The basic addition, subtraction and multiplication in KYBER is over $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$, and the modulus q in KYBER is 3329. This requires modular reduction operation after addition, subtraction and multiplication. There are some universal modular reduction methods, such as Barrett algorithm [13]. But these algorithms consume too much hardware resource.

NTT is a variant of the discrete Fourier transform defined over a finite field $\mathbb{Z}_q$. Suppose that ω_N is a primitive N th root of unit in $\mathbb{Z}_q$, $\mathbf{a} = (a_0, a_1, \dots, a_{n-1})$ is a vector, whose elements a_i , where $i = 0, 1, 2, \dots, n-1$, are in $\mathbb{Z}_q$, i.e., $\mathbf{a} \in \mathbb{Z}_q^n$. If the vector $\mathbf{A} \in \mathbb{Z}_q^n$ satisfies equation (1), $\mathbf{A}$ is the n -point NTT of the vector $\mathbf{a}$. The INTT is defined as equation (2).

$$A_i = \sum_{j=0}^{N-1} a_j \omega_N^{ij} \mod p, (i = 0, 1, \dots, n-1) \quad (1)$$

$$a_i = \frac{1}{n} \sum_{j=0}^{N-1} A_j \omega_N^{-ij} \mod p, (i = 0, 1, \dots, n-1). \quad (2)$$

There are two different methods for NTT calculation, Cooley-Turkey (CT) [11] or Gentleman-Sande (GS) [12].

B. HASH Module

KYBER requires SHA-3 as pseudo-random number generator. Keccak core is a basic unit for SHA-3, and it is hardware-friendly. Keccak-f[1600] requires 1600-bit registers, and the Keccak consumes most of the resource in KYBER.

Manuscript received May 6, 2021; revised June 23, 2021 and July 31, 2021; accepted August 1, 2021. Date of publication August 9, 2021; date of current version March 15, 2022. This work was supported by the National Natural Science Foundation of China under Grant 61974083. This brief was recommended by Associate Editor A. J. Acosta. (Corresponding author: Shuguo Li.)

The authors are with the School of Integrated Circuits, Tsinghua University, Beijing 100084, China (e-mail: gwb.eip@foxmail.com; lishg@tsinghua.edu.cn; kongl17@mails.tsinghua.edu.cn).

Digital Object Identifier 10.1109/TCSII.2021.3103184

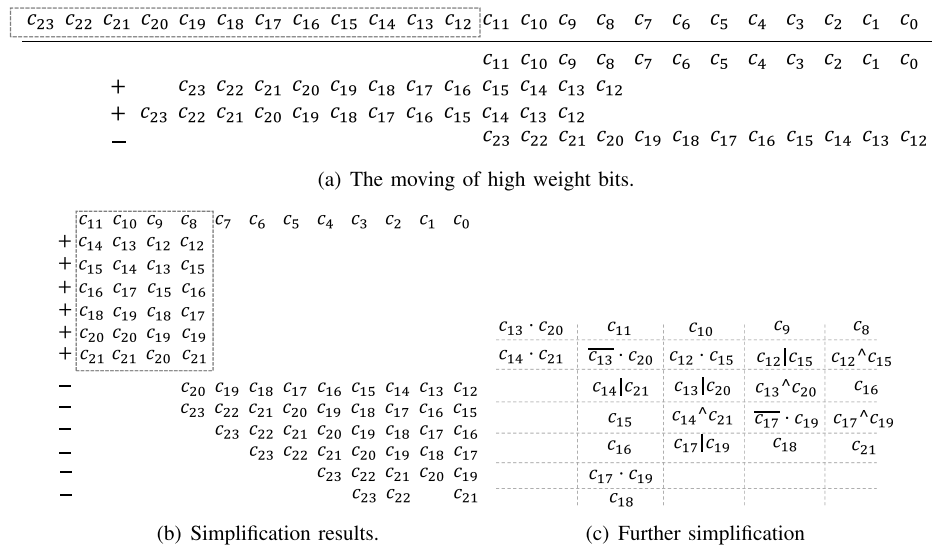


Fig. 1. Modular reduction for $q = 3329$.

III. PROPOSED ARCHITECTURE

A. Modular Reduction in KYBER

The modulus in KYBER is $q = 3329$ which can be represented as a 12-bit unsigned number. Suppose $a, b \in \mathbb{Z}_q$, then their product $c = a \times b$ is a 24-bit number. d, e are the first 12 bits and the last 12 bits of c , namely $c = d \times 2^{12} + e$. Notice that $2^{12} \equiv 3 \times 2^8 - 1 \pmod{3329}$, thus c satisfies

$$\begin{aligned} c &\equiv d \times 2^{12} + e \\ &\equiv (2 + 1) \cdot d \times 2^8 - d + e \pmod{3329} \end{aligned} \quad (3)$$

In Fig. 1, c_i represents the i th bit of c , i.e., $c = \sum_{i=0}^{23} c_i \cdot 2^i$. $c_i \cdot c_j$, $c_i | c_j$ and $c_i \wedge c_j$ represent logical “AND”, “OR” and “XOR” between c_i and c_j , respectively. Fig. 1(a) shows how equation (3) works when performing modular reduction. The bit which has the weight no less than 2^{12} (in the dashed box in Fig. 1(a)) are moved to lower weights. Due to the coefficient “2+1” in equation (3), there are two rows in Fig. 1(a) that start with “+”, one row for 1 times of d and the other for 2 times of d . If we repeatedly use equation (3) to move the bits which have the weight no less than 2^{12} to lower weights, there will be no bits having a weight no less than 2^{12} after a limited number of times. Then, by combining the same bits in the same column, we can get what is shown in Fig. 1(b). In Fig. 1(b), there are some special combinations. For example, the columns that have 2^8 and 2^9 weights both have c_{12} and c_{15} , and the summation of them can be expressed by a simple logical expression, i.e., $\{c_{12}, c_{12}\} + \{c_{15}, c_{15}\} = \{c_{12} \cdot c_{15}, c_{12} | c_{15}\}$. Thus, the summation array in Fig. 1(b) in the dashed box is further simplified as Fig. 1(c) shows. The subtraction array in Fig. 1(b) can also be simplified in the same way.

After the simplification, the summation array and the subtraction array can be added up, then the difference of them is the modular reduction result. Note that there will be some bits which have no less than 2^{12} weights when add the summation array up. We can repeatedly use equation (3) in a limited number of times to solve this problem.

Algorithm 1 Proposed Modular Reduction for 3329

Input: 24-bit number $c \in [0, q^2)$, 12-bit number $q = 3329$

Output: $z \equiv c \pmod{q}$

1: $u = A\text{SumAU}(c)$ 2: $v = ASubAU(c)$

3: $w = u - v$

4: **if** $w < 0$ **then**

5: $z = w + q$

6: **else if** $w \geq q$ **then**
$$7: \quad z = w - q$$
8: **else**

9: $z = w$

10: **end if**

```

11: return  $z$ 

```

Algorithm 1 shows the proposed modular reduction method. Function *ASumAU()* and *ASubAU()* indicate “Add Summation Array Up” and “Add Subtraction Array Up”, respectively.

B. Calculations Related to Polynomials

1) *Number Theoretic Transform*: According to the definition in [3], there are two independent parts of a polynomial $\mathbf{a}(x) = (a_0, a_1, \dots, a_{255})$ when performing the specific NTT in KYBER, namely the even subscript part (denoted as $\mathbf{f}(x) = (a_0, a_2, \dots, a_{254})$) and the odd subscript part (denoted as $\mathbf{g}(x) = (a_1, a_3, \dots, a_{255})$). Thus, the NTT of polynomial $\mathbf{a}(x)$ needs to perform two 128-point NTT which include 7 rounds. Notice that the NTT of $\mathbf{f}(x)$ and $\mathbf{g}(x)$ share the same memory address and twiddle factor, so we distribute two Butterfly Units (BFU) for polynomial $\mathbf{a}(x)$ to calculate the 128-point NTT of $\mathbf{f}(x)$ and $\mathbf{g}(x)$.

For each 128-point NTT, an in-place method we proposed before [16] is used for saving memory usage, but the in-place method will produce a problem that the address of data is generated before “reading”. It should be stored in the register all the time in the process of data calculation, and then it is

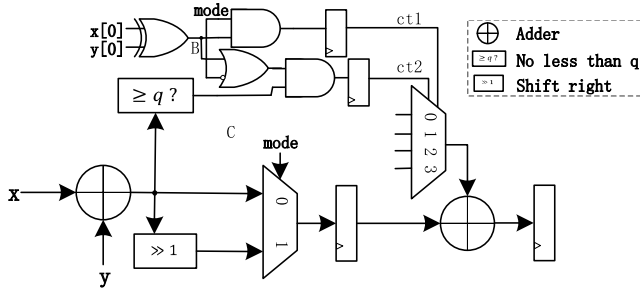


Fig. 2. Modified modular adder.

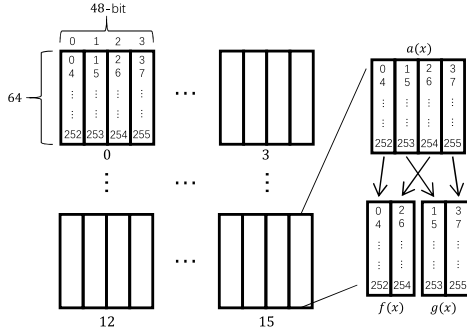


Fig. 3. Organization of the SRAM bank.

used for data write-back. This will consume a lot of register resources. The address generator (AG) we proposed in [16] saves 40% area and has a short critical path, so we use two AGs in this design, one for data-reading and the other for data write-back. This method saves many registers. We also use the Low-Complexity Negative Wrapped Convolution (NWC) [4] which requires 448 independent butterfly operations for both NTT and INTT. The Low-Complexity INTT proposed in [4] requires additional “ $\times \frac{1}{2}$ ” operation in a BFU. Suppose x and y are the inputs of an INTT BFU, and X and Y are corresponding outputs, then, according to [4]

$$\begin{aligned} X &= \frac{x+y}{2} \mod q \\ Y &= \frac{x-y}{2} \omega_m^{-j} \gamma_{2m}^{-1} \mod q. \end{aligned} \quad (4)$$

We precomputed and stored $\frac{\omega_m^{-j} \gamma_{2m}^{-1}}{2}$ to eliminate one “ $\times \frac{1}{2}$ ” operation in the INTT BFU. Further, we support the remaining “ $\times \frac{1}{2}$ ” operation by using a specially modified modular adder in Fig. 2. Suppose that $z = x + y$, if z is even, then $\frac{z}{2} \equiv z \gg 1 \mod q$ ($z \gg 1$ represents the arithmetic shift right of z), and if z is odd, $\frac{z}{2} \equiv z \gg 1 + \frac{q+1}{2} \mod q$. Thus, we have the architecture of modified modular adder in Fig. 2. $x[0]$ and $y[0]$ are the least significant weight bits of x and y , respectively. When the “mode” is 0, the circuit performs $x + y \mod q$, otherwise it performs $\frac{x+y}{2} \mod q$. The four-choice-one multiplexer in Fig. 2 will chose 0, $-q$, $\frac{q+1}{2}$ and $\frac{1-q}{2}$ when {ct1, ct2} equals 00₍₂₎, 01₍₂₎, 10₍₂₎ and 11₍₂₎, respectively. This method avoids extra separate “ $\times \frac{1}{2}$ ” hardware circuits and no additional pipeline stage is needed to compute “ $\times \frac{1}{2}$ ”.

2) *Point-Wise Multiplication*: The point-wise multiplication (PWM) in KYBER [3] requires 5 multiplications, but [5]

adopts Karatsuba algorithm in it, and the new method only requires 4 multiplications. If we combine four modular multipliers, four modular adders and one modular subtractor to perform the PWM in pipeline mode, this structure only consumes 128 clock cycles (or slightly more due to pipeline filling).

3) *Polynomial Addition/Subtraction*: Polynomial addition and subtraction operations in KYBER can be easily done by modular adder and subtractor.

4) *Overall Data Path*: All the calculations related to polynomials mentioned above are performed on the data path. The overall data path contains 12 calculation units: four modular multipliers, four modular adders and four modular subtractors, and they are controlled by some MUXs to perform different calculations, as shown in Fig. 4. When performing NTT or INTT, the 12 calculation units are combined into four BFUs in GS mode or CT mode. These four BFUs share the same cell address (cell address is explained in Section III-D) and the twiddle factor. When performing PWM, the four modular multipliers, three of the modular adders and one modular subtractor are combined. The proposed design requires 457 clock cycles (CC) to perform two polynomials’ NTT or INTT, 140 CCs to perform one polynomial’s PWM and 70 CCs for one polynomial’s addition/subtraction.

C. HASH Module

KYBER requires SHA-3 standard to sample, and the Keccak is a hardware-friendly design. Here we utilize the opensource low throughput Keccak core mentioned in [4], and we configure it to satisfy the sampling procedure. Specific to CBD and reject sampling [3], four CBD cores and four Parse cores are used to deal with the 1600-bit output. The CBD sampling and reject sampling for a single polynomial require approximate 90 and 231 CCs, respectively.

D. Memory Organization

In order to ensure that the sampling result and the intermediate data can be stored and meet the memory access bandwidth required by the data path, the SRAM bank is organized as Fig. 3 shows. There are in total 16 SRAM blocks in the SRAM bank, and each SRAM block has four SRAM cells. The numbers in the cells are the index of coefficients of a polynomial, such as the polynomial $a(x)$ in Section III-B. The depth of the cells is 64 and the bit width is 12, so that a block can hold the coefficients of a polynomial like $a(x)$. According to this polynomial storage method, $f(x)$ and $g(x)$ mentioned in Section III-B are naturally separated as shown on the right side of Fig. 3, so two coefficients of polynomial $f(x)$ or $g(x)$ can be retrieved by one single access to a cell. The retrieved coefficients from a cell are sent to two BFUs to perform two in-place NTT or INTT. Note that there are four BFUs in data path, so two blocks are accessed simultaneously.

As for PWM, there are also two blocks are accessed simultaneously, but the blocks are accessed once every two clock cycles. This is because the eight coefficients retrieved in one block access should be consumed in two clock cycles. If we name the first block’s retrieved coefficients as a_0, a_1, a_2, a_3

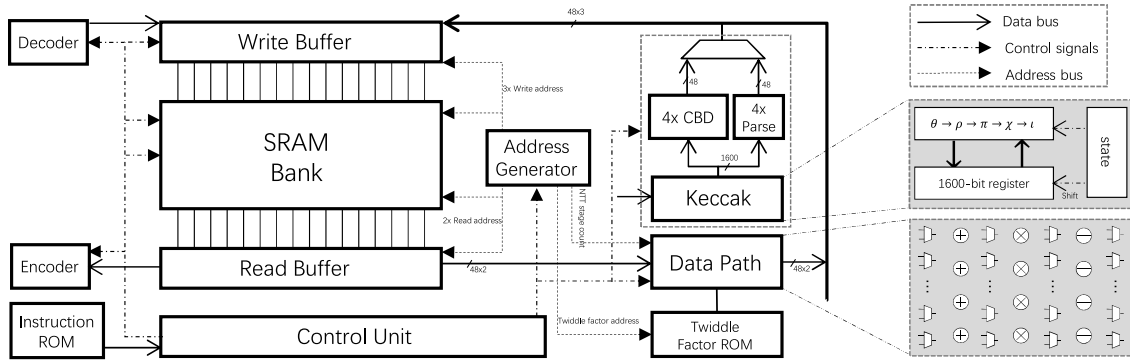


Fig. 4. Overall architecture.

and the second block's as b_0, b_1, b_2, b_3 , then in the first clock cycle a_0, a_1 and b_0, b_1 are used and in the second clock cycle a_2, a_3 and b_2, b_3 are used.

Polynomial addition and subtraction are similar to PWM, but the eight coefficients are consumed in one clock cycle.

The HASH module only needs to write data to SRAM bank. So the SRAM bank needs to read two blocks and write three blocks simultaneously in maximum, as shown in Fig. 4. The total textcoloredSRAM storage is 6kB.

E. Instruction Codes for Describing the Protocol

To simplify the control of the data path and HASH module, we defined a set of instruction codes to perform the KYBER protocol. Each instruction code is 26-bit wide: (1) Three bits for the mode of data path; (2) Three bits for HASH module's mode; (3) Four bits to indicate which SRAM block should the HASH module write to; (4) Next twelve bits are separated into three groups, and each group has four bits, we name them as "r1", "r2" and "wd". When performing NTT/INTT, the data are read from "r1", "r2" and the result are written back to "r1", "r2" (Note that the NTT/INTT are in-place), and when performing PWM, polynomial addition/subtraction, data are read from "r1", "r2" and are written back to "wd". (5) The last four bits are separated into two groups, and each of them contains two bits. We name them as i and j . These two parameters are utilized to sample the matrix A in CPAPKE.KeyGen and CPAPKE.Enc [3].

F. Overall Architecture

The overall architecture is shown in Fig. 4. The address generator generates read/write addresses and "stage count" signal for the SRAM cells and the data path, and it also generate "write enable" and "read enable" signal for every SRAM block according to the input instructions. The read buffer selects two 48-bit outputs from 16 SRAM blocks, and send them to data path for calculation. Data path will generate two 48-bit outputs after performing NTT/INTT, PWM or polynomial addition/subtraction. The 1600-bit output of Keccak core is sent to "CBD" (to perform Binomial Sampling in KYBER) or "Parse" (to perform Rejection Sampling in KYBER) to generate sampling results, and put the results to write buffer. The three 48-bit data in write buffer will be written into SRAM

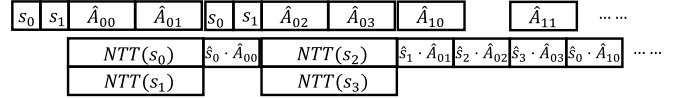


Fig. 5. Overlapping in keygen.

bank according to the write address and write enable signals generated by address generator.

IV. EXPERIMENTAL RESULTS AND COMPARISON

A. Experimental Results

Table II shows the synthesis result on a Xilinx Artix-7 FPGA device with a 159MHz clock. On FPGA platform, SRAMs are implemented with BRAMs and 12-bit multipliers are implemented with DSPs. KYBER-1024, 768 and 512 can be realized by the same hardware design, but due to the length limitation of the article, we only discuss the KYBER-1024 protocol, and different protocols can be realized through different instruction combinations. The proposed design consumes 2253 slices in total, and the HASH module consumes 62% of the total resource. The data path which contains four BFUs consumes 635 slices.

The HASH module and data path are working in parallel, so their execution time can be partially overlapped to reduce the total execution time. Fig. 5 shows how the hash module and the data path works when performing KeyGen [3]. The proposed design needs 7.8k, 8.4k and 10.5k CCs for KeyGen, KENC and KDEC, respectively. In practice, when performing KeyGen, ENC and DEC [3], 12 of the SRAM blocks are enough to store all the intermediate variables.

B. Comparison With Related Works

According to Table II, the proposed modular reduction algorithm needs 142 LUTs and 79 FFs. Compared with the algorithm in [5] which needs 135 LUTs and 96 FFs, our design has 4.9% more LUTs and 17.7% fewer FFs. Our design performs two NTTs/INTTs for two polynomials and the design in [5] performs NTT/INTT for one polynomial. If we double the resource consumption in [5] to perform two polynomial operations, it needs $708 \times 2 = 1416$ LUTs and $574 \times 2 = 1148$ FFs. Then compare our design with the

TABLE I
COMPARISON WITH RELATED WORKS ON FPGA PLATFORM

Work	Platform	CCs [k]	Freq. [MHz]	Time [μ s]	Area				
					LUT	FF	Slice	BRAM	DSP
[5]	Artix-7	9.4/11.3/13.9	161	58.2/67.9/86.2	7.4k	4.6k	2.1k	3	2
[10]	Virtex-7	-/107/135	192	-/557.3/703.1	133k	-	-	548	202
[14]	Artix-7	-/5.7/7.4	210	-/27.4/35.2	12.2k	12.4k	4.5k	15	8
[15]	XCZU9EG	9.0/10.3/12.3	150	60.2/68.4/82.0	24.9k	10.7k	-	2	0
This work	Artix-7	7.8/8.4/10.5	159	49.1/52.8/66.0	7.9k	3.9k	2.3k	16	4

TABLE II
RESOURCE CONSUMPTION OF THE MAJOR MODULE OF KYBER

Module	LUT	FF	BRAM	DSP	Slice
Data Path	1549	788	0	4	635
MR	142	79	0	0	57
HASH	4614	1771	0	0	1407
Keccak	4026	1625	0	0	1056
total	7893	3917	16	4	2253

doubled design in [5], our design has 9.4% more LUTs and 31.3% fewer FFs.

Table I shows the results of the proposed design and other related works. The three columns of “CCs” and “Time” are corresponding to “Key generation” (KeyGen), “Key encapsulation” (KENC) and “Key decapsulation” (KDEC). The proposed design consumes nearly the same slices with [5], but our design requires four DSPs and sixteen BRAM blocks to improve the parallelism, and result in consuming 17.0%, 25.6% and 24.4% fewer CCs when performing KeyGen, KENC and KDEC, respectively. Besides, our design achieves 19% less area-time-product (ATP, slice for area and KENC time for time.) The design in [14] requires 29.5%-32.1% fewer CCs than our design, but our design requires 48.9% fewer slices, 50% fewer DSPs and 218% fewer registers than [14]. Besides, the design in [14] assumes the KeyGen procedure is performed in software. Reference [15] implemented the lattice-based protocol Saber, and this protocol is based on the Learning-with-Rounding (LWR) problem. The highest security level in Saber in [15] requires nearly three times resource than our design but it needs more CCs. This is mainly because NTT is not directly applicable in Saber.

V. CONCLUSION

In this work, we present a highly efficient pure hardware implementation of CRYSTALS-KYBER of the NIST round 3. Our design implemented HASH module, NTT/INTT, PWM and other polynomial related operations, and it is synthesized, as NIST recommended, on a Xilinx Artix-7 FPGA. Using our proposed efficient modular reduction algorithm, reconfigurable data path, memory organization, and merging the “ $\times \frac{1}{2}$ ” operation in $\mathbb{Z}_q$ into modular adder, the proposed design achieves 49.1/52.8/66.0 μ s delay when performing KeyGen/KENC/KDEC, respectively, and it consumes 7.9k LUTs, 3.6k FFs, 2.3k slices, 4 DSPs and 16 BRAMs.

REFERENCES

- [1] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proc. 35th Annu. Symp. Found. Comput. Sci.*, Santa Fe, NM, USA, Nov. 1994, pp. 124–134.
- [2] *Status Report on the Second Round of the NIST Post-Quantum Cryptography Standardization Process*, document NISTIR 8309, Nat. Inst. Stand. Technol., Gaithersburg, MD, USA, 2020.
- [3] R. Avanzi *et al.*, *CRYSTALS-KYBER: Algorithm Specification and Supporting Documentation (Version 3.0). Submission to the NIST Post-Quantum Cryptography Standardization Project*, NIST, Gaithersburg, MD, USA, 2020.
- [4] N. Zhang, B. Yang, C. Chen, S. Yin, S. Wei, and L. Liu, “Highly efficient architecture of NewHope-NIST on FPGA using low-complexity NTT/INTT,” *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2020, no. 2, pp. 49–72, 2020.
- [5] Y. Xing and S. Li, “A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-KYBER on FPGA,” *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2021, no. 2, pp. 328–356, 2021.
- [6] A. Langlois and D. Stehle, “Worst-case to average-case reductions for module lattices,” *Designs Codes Cryptography*, vol. 75, no. 3, pp. 565–599, 2015.
- [7] L. Botros, M. J. Kannwischer, and P. Schwabe, “Memory-efficient high-speed implementation of kyber on Cortex-M4,” in *Progr. 11th Int. Conf. Cryptol. Africa Cryptol. AFRICACRYPT*, Rabat, Morocco, Jul. 2019, pp. 209–228.
- [8] U. Banerjee, T. S. Ukyab, and A. P. Chandrakasan, “Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols,” *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2019, no. 4, pp. 17–61, 2019.
- [9] E. Alkim, H. Evkan, N. Lahr, R. Niederhagen, and R. Petri, “ISA extensions for finite field arithmetic accelerating kyber and newhope on RISC-V,” *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2020, no. 3, pp. 219–242, 2020.
- [10] Y. Huang, M. Huang, Z. Lei, and J. Wu, “A pure hardware implementation of crystals-kyber pqc algorithm through resource reuse,” *IEICE Electron. Exp.*, vol. 17, no. 17, p. 234, 2020.
- [11] J. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Math. Comput.*, vol. 19, pp. 297–301, 1965.
- [12] W. M. Gentleman and G. Sande, “Fast Fourier transforms: For fun and profit,” in *Proc. Amer. Feder. Inf. Process. Soc. AFIPS Fall Joint Comput. Conf.*, San Francisco, CA, USA, Nov. 1966, pp. 563–578.
- [13] P. Barrett, “Implementing the rivest shamir and adleman public key encryption algorithm on a standard digital signal processor,” in *Proc. Adv. Cryptol. CRYPTO*, Santa Barbara, CA, USA, 1986, pp. 311–323.
- [14] V. B. Dang, F. Farahmand, M. Andrzejczak, K. Mohajerani, D. Nguyen, and K. Gaj, “Implementation and benchmarking of round 2 candidates in the NIST post-quantum cryptography standardization process using hardware and software/hardware co-design approaches,” *IACR Cryptol. ePrint Arch.*, Rep. 795, 2020.
- [15] S. S. Roy and A. Basso, “High-speed instruction-set coprocessor for lattice-based key encapsulation mechanism: Saber in hardware,” *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2020, no. 4, pp. 443–466, 2020.
- [16] W. Guo and S. Li, “Area-efficient modular reduction structure and memory access scheme for NTT,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2021, pp. 1–5, doi: [10.1109/ISCAS51556.2021.9401571](https://doi.org/10.1109/ISCAS51556.2021.9401571).